MERN STACK

DAY – 35

4 August 2026

PROJECT: PARKEASY - PARKING MANAGEMENT SYSTEM

BOOK SLOT AND MY BOOKINGS MODULE DEVELOPMENT

• The Book Slot and My Bookings modules are developed to manage the complete parking reservation process in the ParkEasy parking management system.
• The Book Slot module allows users to select a parking slot, choose a booking date and time, and send a reservation request.
• The My Bookings module allows users to view and manage their parking reservations after booking.
• These modules are integrated with the user account system and store booking information inside the user's booking records in MongoDB.

• The Book Slot module collects the following booking details:

1. Parking Slot Number
2. Vehicle Number
3. Booking Date
4. Start Time
5. End Time
6. Booking Status

• The booking request is initially created with Pending status and can be approved or rejected by the administrator.

• Before creating a booking, the system checks:

1. User vehicle details
2. Slot availability

3. Existing bookings for the selected time
4. Vehicle booking conflicts

• The My Bookings module displays all user booking records based on their current status:

1. Pending Requests
2. Approved Bookings
3. Completed Bookings
4. Cancelled / Rejected Bookings

• Each booking record displays:

1. Slot Number
2. Booking Date
3. Booking Time
4. Vehicle Details
5. Booking Status

• Users can cancel pending booking requests, and the system updates the booking status and manages slot availability automatically.

BOOK SLOT API DEVELOPMENT

• The Book Slot API is developed using Express.js to process parking reservation requests from users.
• The API receives booking information from the frontend and stores the booking details in the user's booking array in MongoDB.

API Endpoint

PUT /book-slot/:userId

• The API retrieves the user details using the provided user ID.

User.findById(req.params.userId)

• The API checks existing bookings to prevent:

1. Same slot booking during the same time period
2. Same vehicle multiple bookings at the same time

• If no conflict exists, a new booking object is created.

```
app.put("/book-slot/:userId",async(req,res)=>{
try{
const user=await User.findById(req.params.userId);
if(!user){
return res.json({
success:false,
message:"User not found"
});
}
const vehicleNumber=user.vehicle?.number;
if(!vehicleNumber){
return res.json({
success:false,
message:"Please add vehicle details first"
});
}
const {
slot,
date,
startTime,
endTime
}=req.body;
if(startTime >= endTime){
return res.json({
success:false,
message:"Invalid time range"
});
}
const users=await User.find();
let slotConflict=false;
let vehicleConflict=false;
users.forEach(u=>{
u.bookings.forEach(b=>{
if(
b.status!=="Cancelled" &&
b.status!=="Rejected" &&
b.status!=="Completed"
){
// SAME SLOT TIME CHECK

if(
b.slot===slot &&
b.date===date &&
(
startTime < b.endTime &&
endTime > b.startTime
)
){
slotConflict=true;
}
// SAME VEHICLE TIME CHECK
```

```
if(
b.vehicle===vehicleNumber &&
b.date===date &&
(
startTime < b.endTime &&
endTime > b.startTime
)
){
vehicleConflict=true;
}
}
});
});
if(slotConflict){
return res.json({
success:false,
message:"Slot already booked for selected time"
});
}
if(vehicleConflict){
return res.json({
success:false,
message:"This vehicle already has parking during this time"
});
}
const booking={
slot,
vehicle:vehicleNumber,
date,
startTime,
endTime,
time:`${startTime} - ${endTime}`,
status:"Pending",
entryTime:"",
exitTime:"",
parkingStatus:"Not Entered",
duration:0
};
user.bookings.push(booking);
await user.save();
res.json({
success:true,
message:"Booking request sent successfully"
});
}
catch(err){
res.status(500).json({
success:false,
message:err.message
});
}
});
```

MY BOOKINGS API DEVELOPMENT

• The My Bookings API is developed to retrieve user-specific booking information from MongoDB.

• The API fetches all booking records stored for the logged-in user.

GET /my-bookings/:id

• The API retrieves user details using:

User.findById(req.params.id)

• The response contains:

1. User Name
2. Booking Records

• The frontend displays the received booking data by separating bookings according to their status.

```
app.get("/my-bookings/:id",async(req,res)=>{

try{

const user=await User.findById(req.params.id);
res.json({
name:user.name,
bookings:user.bookings
});
}
catch(err){
res.json({
message:err.message
});

}

});
```

CANCEL BOOKING API DEVELOPMENT

• The Cancel Booking API allows users to cancel their pending parking requests.
• It updates the selected booking status and restores the parking slot availability if required.

PUT /cancel-booking/:userId/:index

• The API receives:

1. User ID
2. Booking Index

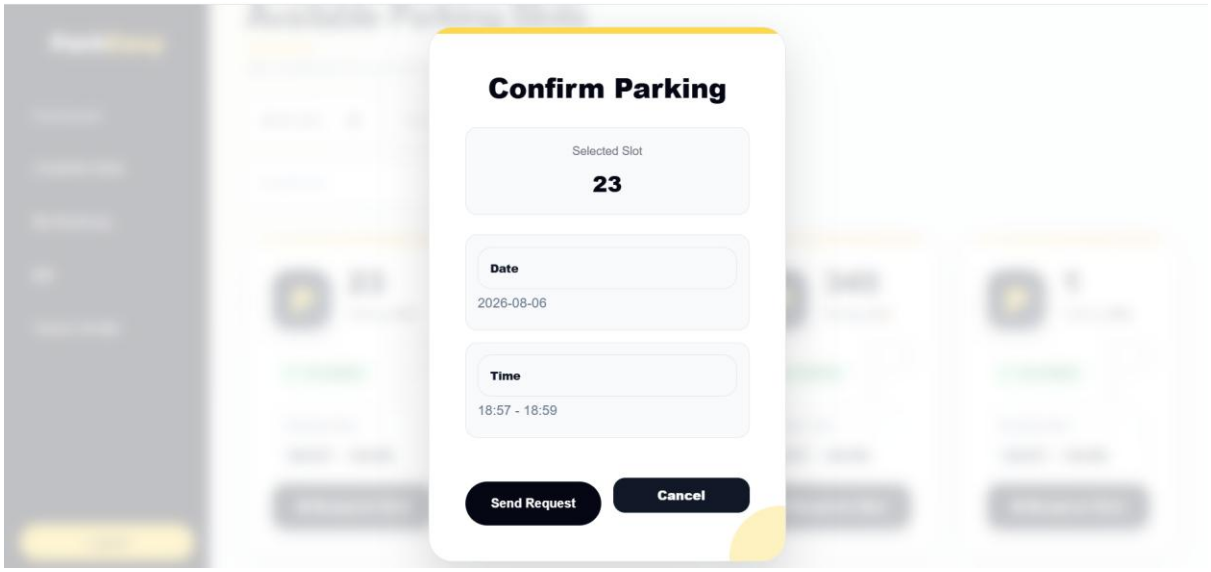
• The selected booking status is updated to: Cancelled

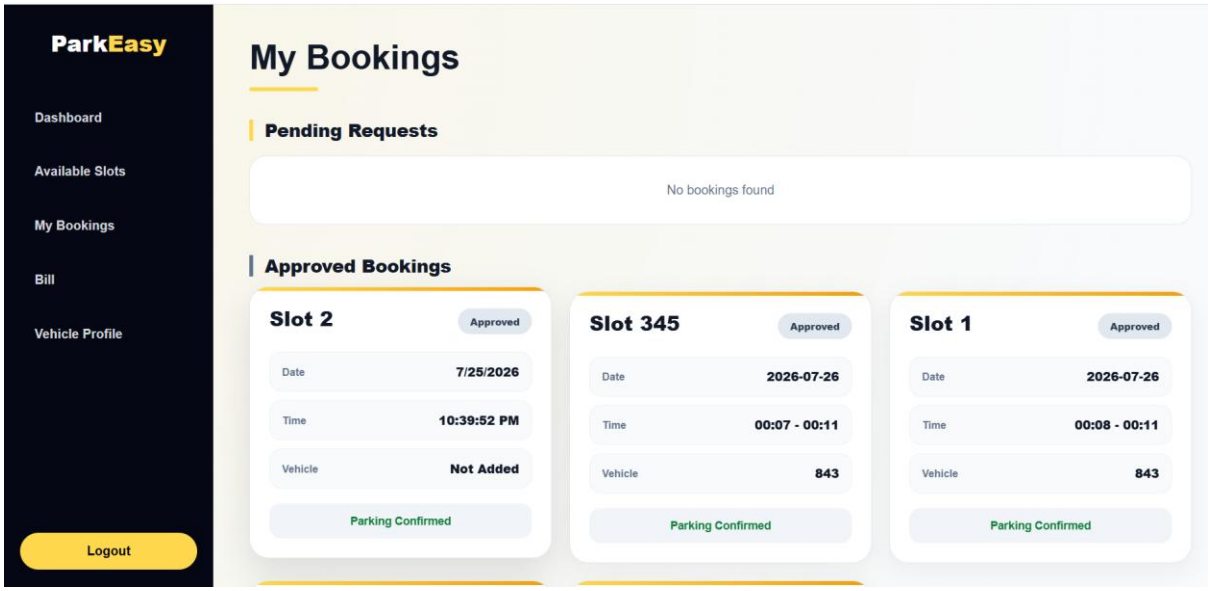
• After cancellation, the system checks whether any other active booking exists for the same slot.
• If no active booking is found, the parking slot status is changed back to Available.

Working

1. The user selects an available parking slot from the Available Slots module.
2. The user provides booking date, start time, and end time details.
3. The frontend sends the booking request using the /book-slot/:userId API.
4. The backend verifies user details, vehicle information, and slot availability.
5. If the booking is valid, the request is stored in MongoDB with Pending status.
6. The user can view all booking records through the My Bookings page.
7. The My Bookings component fetches booking data using the /my-bookings/:id API.
8. Booking records are displayed according to their status.
9. Users can cancel pending requests using the Cancel Booking option.
10. The cancellation request updates the booking status and refreshes the booking list.

```
app.get("/my-bookings/:id",async(req,res)=>{ ···
});
app.put("/cancel-booking/:userId/:index",
async(req,res)=>{
try{
const user=await User.findById(
req.params.userId
);
const booking=user.bookings[
req.params.index
];
booking.status="Cancelled";
const slot=await Slot.findOne({
slotNumber:booking.slot
});
if(slot && slot.status!=="Maintenance"){
const otherBooking = await User.find({
bookings:{
$elemMatch:{
slot:booking.slot,
status:{
$in:[
"Pending",
"Approved"
]
}
}
}
});
if(otherBooking.length===0){
slot.status="Available";
await slot.save();
}
await slot.save();
}
await user.save();
res.json({
message:"Booking cancelled"
});
}
catch(err){
res.status(500).json({
message:err.message
});
}
});
```


Confirm Parking
Selected Slot
23
Date
2026-08-06
Time
18:57 - 18:59
Send Request
Cancel


ParkEasy
Dashboard
Available Slots
My Bookings
Bill
Vehicle Profile
Logout
My Bookings
Pending Requests
No bookings found
Approved Bookings
Slot 2
Approved
Date
7/25/2026
Time
10:39:52 PM
Vehicle
Not Added
Parking Confirmed
Slot 345
Approved
Date
2026-07-26
Time
00:07 - 00:11
Vehicle
843
Parking Confirmed
Slot 1
Approved
Date
2026-07-26
Time
00:08 - 00:11
Vehicle
843
Parking Confirmed